# Worldview OS

## PRD + System Architecture + Prompt Spec

## 1. Product Requirements Document

### 1.1 Product summary

Worldview OS is a structured intelligence engine that helps users think through complex questions across politics, macroeconomics, markets, crypto, sports, gaming, and other domains. Instead of producing a single elegant answer, it decomposes a question into relevant analytical lenses, generates competing scenarios, scores their probability and impact, identifies leading indicators, and recommends what to watch next.

The product is designed to reduce narrative overconfidence, surface hidden variables, and create repeatable, updateable analysis.

### 1.2 Problem

Most AI analysis tools fail in five ways: 1. They answer vague questions without defining the target variable. 2. They collapse multi-causal situations into a single narrative. 3. They produce prose rather than forecasts. 4. They do not show what evidence would change their view. 5. They do not remember prior forecasts or revisions.

Users who care about decisions, investing, research, strategy, or forecasting need a system that is structured, probabilistic, cross-domain, and updateable.

### 1.3 Vision

Ask any serious question and get the full map of how to think about it:

- what the real question is
- what lenses matter
- what scenarios are live
- what variables drive the answer
- what indicators to monitor
- what would change the odds

### 1.4 Target users

Primary users:

- investors and traders
- operators and founders
- researchers and analysts
- content creators and strategists
- forecasters and bettors

Secondary users:

- policy professionals
- journalists
- advanced hobbyists in sports, gaming, and crypto

## 1.5 Jobs to be done

Users want to:

- turn vague questions into analyzable ones
- understand all serious angles behind an issue
- compare competing explanations and scenarios
- track probabilities over time
- identify underappreciated risks and catalysts
- decide what to monitor next
- create reusable analysis logs and forecast histories

## 1.6 Product principles

1. Define before you predict.
2. Prefer structured reasoning over elegant prose.
3. Show competing scenarios, not one narrative.
4. Expose uncertainty explicitly.
5. Rank variables by causal importance and measurability.
6. Separate evidence from interpretation.
7. Make every forecast updateable.
8. Preserve memory of prior views and revisions.

## 1.7 MVP scope

Launch with three domain packs:

- politics / geopolitics
- macro / markets
- sports

MVP capabilities:

- question normalization
- question type routing
- lens expansion
- driver extraction and ranking
- scenario generation
- evidence for / against
- indicator watchlist
- probability, impact, and confidence scoring
- red-team challenge
- synthesis summary
- forecast log storage

Not in MVP:

- full autonomous monitoring
- broad all-domain coverage
- automatic live data ingestion from dozens of sources
- user-custom fine-tuned domain ontologies
- advanced collaboration workflows

## 1.8 Core use cases

**Use case 1: Political risk analysis**

User asks: “By 2027, what are the main ways Trump’s second-term agenda could materially fail?”
System outputs:

- normalized question
- definitions of failure
- relevant lenses
- top drivers
- scenario table
- leading indicators
- what would change the forecast

**Use case 2: Market scenario analysis**

User asks: “What are the 12-month scenarios for BTC if the Fed cuts late and fiscal deficits stay high?”
System outputs:

- macro lens
- liquidity lens
- regulatory lens
- narrative lens
- scenario probabilities
- catalysts and invalidation signals

**Use case 3: Sports forecasting**

User asks: “What are the strongest arguments for and against the Celtics winning the title?” System outputs:

- roster lens
- injury lens
- playoff matchup lens
- efficiency indicators
- scenario matrix
- hidden variables

## 1.9 User workflow

1. User submits a question.
2. System classifies the question type.
3. System rewrites the question into a precise target variable.

4. System selects relevant domains and lens packs.
5. System generates driver map and scenario set.
6. System evaluates supporting and opposing evidence.
7. System scores scenarios.
8. System returns structured analysis.
9. User saves, revises, or tracks the forecast.

## 1.10 Question types

The system should classify each query into one primary type and optional secondary types.

Primary types:

- forecast: what is likely to happen
- diagnosis: why something happened or is happening
- comparison: compare options, actors, assets, or outcomes
- decision support: what should I do given scenarios
- narrative audit: what stories are overstated or misleading
- risk scan: what could go wrong and how badly

## 1.11 Functional requirements

**FR1. Question normalization**

The system must:

- rewrite vague questions into precise questions
- identify ambiguities in scope, timeframe, and target variable
- define key terms such as fail, succeed, collapse, bullish, overvalued

**FR2. Domain and lens routing**

The system must:

- identify relevant domains
- load universal lens library
- load domain-specific lens packs
- rank lenses by relevance

**FR3. Driver mapping**

The system must identify and rank drivers under these buckets:

- structural drivers
- cyclical drivers
- trigger events
- narrative and reflexive amplifiers

**FR4. Scenario generation**

The system must generate at least four scenarios where appropriate:

- base case
- upside / bullish case
- downside / bearish case
- wildcard / tail case

Each scenario must include:

- description
- probability range
- expected impact
- time horizon
- leading indicators
- disconfirming evidence

**FR5. Evidence discipline**

The system must produce:

- strongest arguments for the thesis
- strongest arguments against the thesis
- hidden variables
- base-rate considerations
- confidence notes

**FR6. Scoring engine**

For each scenario, the system must output:

- probability score
- impact score
- confidence score
- evidence strength score
- update triggers

**FR7. Red-team function**

The system must generate a challenge section that:

- attacks the main conclusion
- identifies missing assumptions
- names the most likely reasoning failure

**FR8. Synthesis**

The system must produce:

- concise bottom line
- top 3 to 5 variables to watch

- what would change its mind
- optional action layer by user profile

**FR9. Memory and forecast tracking**

The system must store:

- the original question
- normalized question
- scenario probabilities
- assumptions
- indicators
- revisions
- revision reasons
- realized outcomes when available

## 1.12 Non-functional requirements

- structured outputs in JSON and human-readable markdown
- deterministic format even when content varies
- low hallucination tendency through explicit evidence framing
- modular domain packs
- fast enough for interactive use
- clear audit trail for revisions

## 1.13 Success metrics

Product metrics:

- percentage of outputs with fully populated scenario matrix
- percentage of outputs with explicit definitions
- save rate of analyses
- repeat usage per user
- forecast revision rate
- user-rated usefulness

Quality metrics:

- calibration of probability forecasts
- forecast Brier score or equivalent where measurable
- rate of user-detected missing drivers
- rate of contradictory outputs on repeated similar inputs

## 1.14 Risks

- overproduction of low-value angles
- fake balance in asymmetric situations
- overconfident probabilities without solid evidence
- poor domain routing
- narrative contamination from weak sources
- memory drift and stale assumptions

### 1.15 MVP release criteria

A release is ready when the system can consistently:

- normalize ambiguous questions
- classify question type correctly in most cases
- produce ranked lenses
- generate at least 3 to 4 credible scenarios
- assign coherent scores
- identify indicators and invalidation signals
- store outputs in a forecast log

---

## 2. System Architecture

### 2.1 Architecture overview

Worldview OS should be built as a modular pipeline rather than a monolithic agent. The system has six main layers:

1. Input and orchestration layer
2. World model and ontology layer
3. Analysis engine layer
4. Evidence and retrieval layer
5. Memory and forecast tracking layer
6. Output and interface layer

### 2.2 High-level component diagram

```
User Query
   ↓
API / UI Layer
   ↓
Orchestrator
   ├─ Question Type Router
   ├─ Domain Router
   ├─ Lens Loader
   └─ Workflow Controller
   ↓
Analysis Committee
   ├─ Normalizer Agent
   ├─ Domain Expert Agents
   ├─ Driver Mapper
   ├─ Scenario Engine
   ├─ Red Team Agent
   └─ Synthesizer Agent
   ↓
Evidence + Memory Services
   ├─ Retrieval / Search
```

```
    ├─ Source Ranking
    ├─ Forecast Log
    ├─ User Preferences
    └─ Indicator History
   ↓
Output Formatter
    ├─ JSON schema output
    └─ Human-readable report
```

## 2.3 Layer details

**Layer 1: Input and orchestration**

Responsibilities:

- accept user query and metadata
- identify user profile and desired objective
- route workflow
- enforce output schema
- decide when to invoke domain experts

Inputs:

- raw question
- optional domain
- optional time horizon
- optional user objective
- optional user type

Outputs:

- workflow plan
- selected domains
- selected lens packs
- required analysis components

**Layer 2: World model and ontology**

This is the system’s map of entities, domains, forces, and mechanisms.

Core ontology objects:

- Domain
- Actor
- Force
- Mechanism
- Indicator
- Scenario
- Claim
- Evidence
- Forecast

Example entities:

- domains: politics, macro, markets, crypto, sports
- actors: governments, central banks, teams, firms, regulators, players
- forces: liquidity, incentives, demographics, conflict, narrative, regulation
- mechanisms: rate cuts affect liquidity, injuries affect team strength, executive action affects policy outcomes

Purpose:

- provide shared vocabulary across agents
- allow lens selection and driver mapping
- improve consistency and memory retrieval

**Layer 3: Analysis engine**

The analysis engine is a committee, not one agent.

Core modules: 1. Question Type Router 2. Normalizer 3. Lens Expander 4. Driver Mapper 5. Scenario Engine 6. Evidence Evaluator 7. Red Team 8. Synthesizer

Recommended committee roles:

- Political Analyst
- Macro Strategist
- Market Analyst
- Domain Specialist
- Historian / Base-Rate Analyst
- Behavioral / Narrative Analyst
- Red Team Skeptic
- Probability Forecaster
- Synthesizer

Each module should have a narrow responsibility and a structured output contract.

**Layer 4: Evidence and retrieval**

Responsibilities:

- retrieve relevant evidence
- rank sources
- separate direct evidence from inference
- attach evidence to claims and scenarios

Retrieval sources may include:

- news and public information
- market data APIs
- macroeconomic data feeds
- sports statistics APIs
- user-uploaded notes and past analyses

Source ranking principles:

- prefer primary or authoritative sources for facts
- distinguish current data from historical context
- record timestamp and recency
- score source reliability

**Layer 5: Memory and forecast tracking**

Memory should include three subtypes: 1. User memory

- user goals
- preferred domains
- preferred style
- risk appetite

- Analytical memory

- prior questions

- prior assumptions
- prior scenario scores
- prior watchlists

- Forecast memory

- dated forecast versions

- revision reasons
- outcome resolution when known

A forecast log is critical for learning.

**Layer 6: Output and interface**

The system should support two output modes:

- structured machine-readable JSON
- readable report for human users

Views:

- quick synthesis view
- full scenario matrix view
- driver map view
- watchlist view
- revision history view

## 2.4 Data model

### Core schema objects

```
{
 "query": {
  "raw": "string",
  "normalized": "string",
  "question_type": "forecast | diagnosis | comparison | decision_support | narrative_audit |
risk_scan",
  "time_horizon": "string",
  "user_objective": "string"
 },
 "definitions": [
  {
   "term": "string",
   "definition": "string"
  }
 ],
 "domains": ["string"],
 "lenses": [
  {
   "name": "string",
   "relevance_rank": 1,
   "why_it_matters": "string"
  }
 ],
 "drivers": [
  {
   "name": "string",
   "category": "structural | cyclical | trigger | narrative",
   "importance": 1,
   "measurability": 1,
   "directionality": "supports | weakens | mixed"
  }
 ],
 "scenarios": [
  {
   "name": "string",
   "description": "string",
   "probability": 0.0,
   "impact": 0.0,
   "confidence": 0.0,
   "time_horizon": "string",
   "leading_indicators": ["string"],
   "disconfirming_signals": ["string"]
  }
 ],
 "arguments_for": ["string"],
 "arguments_against": ["string"],
 "hidden_variables": ["string"],
```

```
  "what_would_change_my_mind": ["string"],
  "bottom_line": {
    "summary": "string",
    "confidence": 0.0,
    "watchlist": ["string"]
  }
}
```

### 2.5 Scoring design

Use normalized scales for consistency.

Probability:

- 0.00 to 1.00

Impact:

- 1 to 5
- 1 = minor
- 3 = meaningful
- 5 = regime-changing

Confidence:

- 1 to 5
- based on evidence quality, model clarity, and variable stability

Evidence strength:

- 1 to 5
- based on source quality, directness, and consistency

Measurability:

- 1 to 5
- how observable the driver is in real time

To avoid fake precision, the UI can show both exact stored values and rounded display ranges.

### 2.6 Workflow sequence

```
1. Receive raw query
2. Classify question type
3. Normalize question and define target variable
4. Select domains and lens packs
5. Generate and rank drivers
6. Produce candidate scenarios
7. Gather evidence for and against
8. Score scenarios and confidence
```

```
9. Run red-team critique
10. Synthesize final output
11. Save to forecast log
```

## 2.7 Domain pack design

A domain pack contains:

- domain ontology extensions
- domain-specific lens library
- domain-specific indicators
- scoring heuristics
- common failure modes
- default data connectors

Example: Macro / Markets pack

- lenses: rates, liquidity, fiscal policy, earnings, positioning, sentiment, regulation
- indicators: CPI, payrolls, yield curve, credit spreads, dollar index, M2 proxies

Example: Sports pack

- lenses: roster quality, injuries, schedule difficulty, coaching, matchup geometry, morale, market odds
- indicators: net rating, injury reports, rotation stability, rest days, opponent profile

## 2.8 Suggested implementation stack

Backend:

- Python service
- orchestration framework for multi-step reasoning
- PostgreSQL for metadata and forecast logs
- vector store for memory retrieval
- Redis for caching

LLM layer:

- one strong general reasoning model
- optional specialist prompts rather than separate fine-tuned models for MVP

Data sources:

- market and macro APIs
- web/news retrieval
- sports stats APIs
- user uploads and notes

Frontend:

- web app with report and dashboard views
- saved analyses

- indicator watchlists
- revision timeline

### 2.9 Reliability controls

- schema validation on every agent output
- contradiction checks across scenario probabilities
- evidence timestamp checks
- explicit separation of evidence vs inference
- red-team mandatory for high-stakes queries
- calibration review of forecasts over time

---

## 3. Prompt Specification

### 3.1 Prompting philosophy

The system should use a chain of structured prompts with explicit output schemas rather than one giant prompt. Each module should do one thing well and hand off a clean object to the next module.

### 3.2 Global system prompt

```
You are Worldview OS, a structured intelligence engine.

Your job is not to answer immediately with a single narrative.
Your job is to:
1. clarify the real question
2. define key terms
3. identify relevant domains and lenses
4. map the main drivers
5. generate competing scenarios
6. evaluate evidence for and against
7. score probability, impact, and confidence
8. state what would change your mind
9. produce a concise synthesis and watchlist

Rules:
- Prefer structured reasoning over elegant prose.
- Never skip definitions when terms are vague.
- Do not collapse competing possibilities into one story unless justified.
- Be explicit about uncertainty.
- Distinguish facts, inferences, and speculation.
- Rank drivers by causal importance and measurability.
- Include a red-team critique.
- Output must follow the provided schema.
```

### 3.3 Orchestrator prompt

```
You are the Orchestrator.

Input:
- raw user query
- optional domain
- optional time horizon
- optional user objective

Tasks:
1. Classify the primary question type.
2. Identify the relevant domains.
3. Decide which domain packs to load.
4. Decide which analysis modules must run.
5. Return a workflow plan only.

Return JSON with:
- question_type
- domains
- lens_packs
- required_modules
- notes
```

### 3.4 Normalizer prompt

```
You are the Question Normalizer.

Input:
- raw query
- user objective
- time horizon if present

Tasks:
1. Rewrite the question into a precise analyzable form.
2. Identify ambiguities.
3. Define key terms needed for analysis.
4. Specify the target variable.
5. Specify the relevant time horizon.

Return JSON with:
- normalized_question
- target_variable
- ambiguities
- definitions
- time_horizon
```

### 3.5 Lens router prompt

```
You are the Lens Router.

Input:
- normalized question
- domains
- loaded domain packs

Tasks:
1. Select the most relevant analytical lenses.
2. Rank them by relevance.
3. For each lens, explain why it matters.
4. Exclude low-value or redundant lenses.

Use both universal lenses and domain-specific lenses.

Return JSON with:
- selected_lenses: [{name, relevance_rank, why_it_matters}]
```

### 3.6 Driver mapper prompt

```
You are the Driver Mapper.

Input:
- normalized question
- selected lenses

Tasks:
1. Identify the key drivers that move the answer.
2. Categorize each driver as structural, cyclical, trigger, or narrative.
3. Rank each driver by causal importance.
4. Score each driver for measurability.
5. Note whether each driver strengthens, weakens, or ambiguously affects the
thesis.

Return JSON with:
- drivers: [{name, category, importance, measurability, directionality,
rationale}]
- top_variables_to_watch
```

### 3.7 Scenario engine prompt

```
You are the Scenario Engine.

Input:
```

```
- normalized question
- definitions
- selected lenses
- drivers

Tasks:
1. Generate the most credible scenario set.
2. Include base, upside, downside, and wildcard scenarios when appropriate.
3. For each scenario, provide a clear description.
4. Assign a probability.
5. Assign an impact score.
6. Assign a confidence score.
7. List leading indicators.
8. List disconfirming signals.

Rules:
- Scenario probabilities should be coherent as a set.
- Avoid fake precision where evidence is weak.

Return JSON with:
- scenarios: [{name, description, probability, impact, confidence,
time_horizon, leading_indicators, disconfirming_signals}]
```

### 3.8 Evidence evaluator prompt

```
You are the Evidence Evaluator.

Input:
- normalized question
- scenarios
- retrieved evidence

Tasks:
1. Identify strongest arguments supporting the main thesis.
2. Identify strongest arguments against it.
3. Identify hidden variables or underappreciated factors.
4. Assess evidence strength.
5. Distinguish direct evidence from inference.

Return JSON with:
- arguments_for
- arguments_against
- hidden_variables
- evidence_notes
```

### 3.9 Red-team prompt

```
You are the Red Team.

Input:
- normalized question
- drivers
- scenarios
- evidence summary
- draft bottom line

Tasks:
1. Attack the leading conclusion.
2. Identify the most fragile assumptions.
3. Identify what has likely been underweighted.
4. State the most likely reasoning failure.
5. Propose an alternative interpretation.

Return JSON with:
- fragile_assumptions
- missing_factors
- likely_reasoning_failure
- strongest_counterview
```

### 3.10 Synthesizer prompt

```
You are the Synthesizer.

Input:
- normalized question
- definitions
- lenses
- drivers
- scenarios
- arguments_for
- arguments_against
- hidden_variables
- red_team_output

Tasks:
1. Produce the final structured report.
2. Summarize the bottom line in a few sentences.
3. List the top variables to watch.
4. State what would change your mind.
5. Keep the tone analytical, not theatrical.

Return JSON with:
```

```
- bottom_line
- watchlist
- what_would_change_my_mind
- summary_confidence
```

### 3.11 Output format prompt

```
Format the final output in two forms:
1. JSON following the schema exactly.
2. Human-readable markdown with these sections:
   A. Reframed question
   B. Definitions
   C. Relevant lenses
   D. Key drivers
   E. Scenario matrix
   F. Arguments for
   G. Arguments against
   H. Hidden variables
   I. What would change my mind
   J. Bottom line
```

### 3.12 Universal lens library

This should be available to the Lens Router.

Universal lenses:

- structural
- incentive
- power
- timing
- historical analog
- market pricing
- reflexivity
- second-order effects
- fragility
- narrative
- legal / institutional
- human behavior
- base-rate
- tail-risk
- measurable data

### 3.13 Domain pack prompt extension example

**Politics / geopolitics pack**

```
When the domain includes politics or geopolitics, consider these additional
lenses:
- institutional constraints
- coalition stability
- executive capacity
- court and legal constraints
- bureaucratic implementation
- public approval
- media ecosystem
- geopolitical spillovers
- state / local resistance
```

**Macro / markets pack**

```
When the domain includes macro or markets, consider these additional lenses:
- liquidity
- rates and policy path
- fiscal stance
- growth and labor conditions
- inflation dynamics
- valuation and positioning
- regulation
- narrative and sentiment
- cross-asset contagion
```

**Sports pack**

```
When the domain includes sports, consider these additional lenses:
- roster quality
- health and injury risk
- coaching quality
- matchup geometry
- fatigue and schedule
- morale and chemistry
- market odds vs fundamentals
- playoff variance or game-specific variance
```

### 3.14 Example final answer template

```
## Reframed question
[normalized question]
```

```
## Definitions
- [term]: [definition]

## Relevant lenses
1. [lens] — [why it matters]

## Key drivers
1. [driver] — [importance and measurability]

## Scenario matrix
| Scenario | Description | Probability | Impact | Confidence | Leading
Indicators |
|---------|------------|------------|--------|-----------|-------------------|

## Arguments for
- [argument]

## Arguments against
- [argument]

## Hidden variables
- [hidden variable]

## What would change my mind
- [condition]

## Bottom line
[2 to 5 sentence synthesis]
```

---

## 4. Recommended MVP Build Plan

### Phase 1

- single orchestrated workflow
- three domain packs
- JSON schema outputs
- markdown report outputs
- manual save to forecast log

### Phase 2

- evidence retrieval integration
- revision history and watchlists
- user profile memory
- domain-specific scoring heuristics

### Phase 3

- automatic indicator refresh
- alerts on threshold changes
- forecast performance dashboard
- collaborative analysis workspaces

---

## 5. Final recommendation

Do not position Worldview OS as an answer bot. Position it as a structured analysis engine that helps users define the question, see the scenario landscape, and track what matters. The moat is not eloquence. The moat is disciplined decomposition, forecasting structure, indicator tracking, and revision memory.